

# Dokumentacja wstępna — Zadanie 24

Optymalizacja układu VSI z wykorzystaniem strategii ewolucyjnych i ewolucji różnicowej

#2, #3

Wybrane Zagadnienia Algorytmów Ewolucyjnych (WAE), semestr letni 2025/2026

22 kwietnia 2026

## 1 Opis problemu i sposobu rozwiązania

**Kontekst.** Przetwornica typu VSI (*voltage-source inverter*) to układ energoelektroniczny służący do zamiany napięcia stałego na przemienne. Udostępniony symulator w MATLAB-ie opisuje jego działanie w przestrzeni stanów

$$\dot{\mathbf{x}}(t) = \mathbf{A}\mathbf{x}(t) + \mathbf{B}\mathbf{u}(t), \quad \mathbf{y}(t) = \mathbf{C}\mathbf{x}(t) + \mathbf{D}\mathbf{u}(t),$$

a sterowanie realizowane jest za pomocą dyskretnego regulatora liniowo-kwadratowego (DLQ), który minimalizuje funkcjonal

$$J = \sum_{k=0}^{\infty} (\mathbf{x}_k^{\top} \mathbf{Q} \mathbf{x}_k + \mathbf{u}_k^{\top} \mathbf{R} \mathbf{u}_k).$$

W praktyce jakość działania regulatora w dużej mierze zależy od doboru macierzy wagowych  $\mathbf{Q}$  i  $\mathbf{R}$ , które ustala projektant.

**Sformułowanie zadania.** Symulator udostępnia funkcję celu  $f : \mathbb{R}^4 \rightarrow \mathbb{R}$  poprzez interfejs XML-RPC. Wartość tej funkcji odpowiada jakości regulacji (liczonej jako pewien wskaźnik całkowity po symulacji układu w zamkniętej pętli). Traktujemy ją jako tzw. czarną skrzynkę — nie znamy jej postaci analitycznej ani gradientu, a pojedyncza ewaluacja jest stosunkowo kosztowna. Dodatkowo zależność między parametrami a jakością regulacji może być nieliniowa i posiadać wiele minimów lokalnych.

Naszym celem jest znalezienie wektora parametrów  $\mathbf{p}^* = \arg \min_{\mathbf{p} \in \mathcal{P}} f(\mathbf{p})$ , gdzie zbiór  $\mathcal{P}$  uwzględnia ograniczenia fizyczne (np. nieujemność i górne ograniczenia wag). Ograniczenia te będziemy obsługiwać poprzez rzutowanie, odpowiednie przekształcenia zmiennych lub — w razie potrzeby — funkcję kary.

**Metody.** W rozważaniach uwzględniamy trzy populacyjne metody bezgradientowe:

- **$(\mu, \lambda)$ -ES z samoadaptacją  $\sigma$**  — stosunkowo prosty wariant strategii ewolucyjnej, który potraktujemy jako punkt odniesienia. Pozwoli on sprawdzić, jak dużo zyskujemy dzięki bardziej zaawansowanym mechanizmom adaptacji.
- **jDE** (Brest i in., 2006) — samoadaptacyjna wersja ewolucji różnicowej, w której parametry  $F$  i  $CR$  są częścią reprezentacji osobników i zmieniają się w trakcie działania algorytmu. Metoda ta jest znana z dość dobrej odporności na początkowy dobór parametrów.
- **CMA-ES** (*Covariance Matrix Adaptation Evolution Strategy*, Hansen & Ostermeier) — główny algorytm używany w pracy. Uchodzi za jedną z najskuteczniejszych metod dla ciągłych problemów typu czarnej skrzynki, zwłaszcza w niewielkich i średnich wymiarach. Dzięki adaptacji macierzy kowariancji potrafi uchwycić różne skale wpływu poszczególnych parametrów na wartość funkcji celu.

Dodatkowo planujemy porównać powyższe metody z dostępnym w repozytorium algorytmem PSO, aby sprawdzić, czy bardziej zaawansowane mechanizmy adaptacyjne rzeczywiście przekładają się na lepsze wyniki przy ograniczonym budżecie obliczeń.

## 2 Planowane eksperymenty numeryczne

Eksperymenty zostaną przeprowadzone zgodnie z przyjętymi praktykami porównywania metaheurystyk (np. CEC, BBOB):

1. *Budżet obliczeniowy.* Ze względu na koszt pojedynczej symulacji w MATLAB-ie zakładamy wstępnie budżet  $FES_{\max} = 2000\text{--}5000$  ewaluacji na jedno uruchomienie. Dokładna wartość zostanie ustalona po zmierzeniu czasu pojedynczej ewaluacji.
2. *Strojenie parametrów.* Parametry algorytmów (np.  $(\mu, \lambda)$ ,  $\sigma_0$ , rozmiar populacji) zostaną dobrane eksperymentalnie na mniejszym budżecie, z uwzględnieniem różnych ziaren generatora liczb pseudo-losowych.
3. *Porównanie metod.* Każdy algorytm (wraz z PSO) zostanie uruchomiony co najmniej 30 razy dla różnych punktów startowych i ziaren losowych. Analizie poddamy m.in. najlepsze uzyskane wartości funkcji celu, medianę, średnią oraz odchylenie standardowe, a także przebieg konwergencji. Ziarna zostaną zapisane, aby zapewnić odtwarzalność eksperymentów.
4. *Testy statystyczne.* Do porównania wyników użyjemy testów nieparametrycznych (np. Wilcoxona lub Manna–Whitneya), wraz z poprawką Holma na wielokrotne porównania.
5. *Ocena jakości sterowania.* Najlepsze rozwiązania zostaną dodatkowo ocenione jakościowo — przeanalizujemy przebiegi czasowe, przeregulowanie, czas regulacji oraz wskaźniki takie jak THD.
6. *Analiza wrażliwości.* Sprawdzimy, jak zmieniają się wyniki w zależności od budżetu obliczeniowego i rozmiaru populacji, co pozwoli sformułować praktyczne wnioski dotyczące stosowania poszczególnych metod.

## 3 Wybór technologii

- **Simulator:** MATLAB 2025a wraz z wymaganymi dodatkami; komunikacja poprzez XML-RPC.
- **Warstwa optymalizacyjna:** Python  $\geq 3.12$ , z wykorzystaniem NumPy, SciPy oraz `scipy.stats`.
- **Algorytmy:** CMA-ES — biblioteka `pycma`;  $(\mu, \lambda)$ -ES i jDE — implementacje własne, umożliwiające większą kontrolę nad działaniem i porównywalność wyników.
- **Analiza i wizualizacja:** Matplotlib, pandas; raport końcowy przygotowany w L<sup>A</sup>T<sub>E</sub>X.
- **Organizacja pracy:** Git/GitHub, środowiska `venv`, testy optymalizatorów (`pytest` dla testów i `cocoex` dla funkcji BBOB wykorzystywanych w testach), statyczna analiza kodu i formatowanie (`ruff`); wyniki zapisywane w formatach CSV.